UWE Bristol | University of the West of England

Lecture 5: System analysis and design process cont. + SOLID principles

Week 5

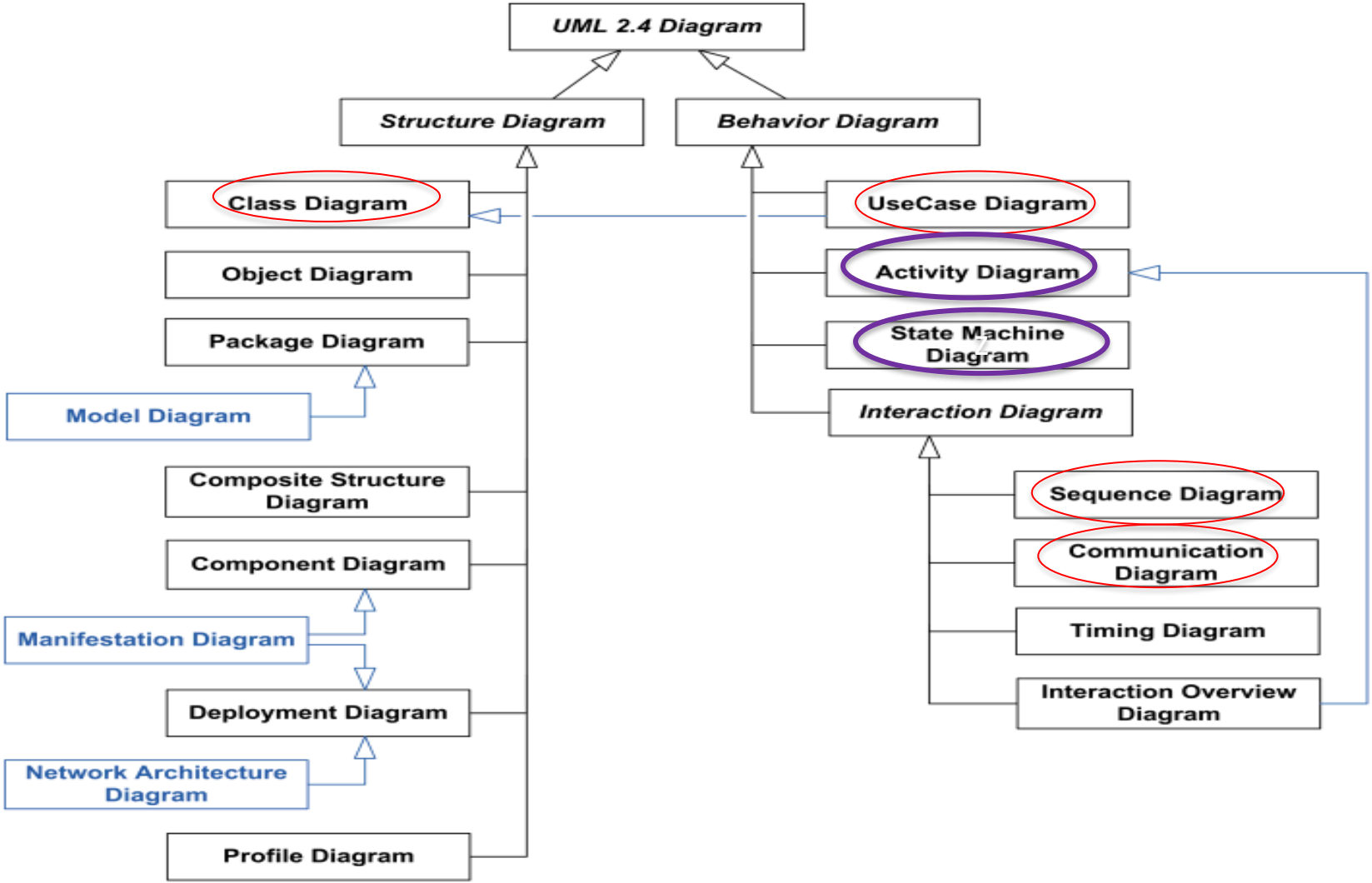
Outline

–UCD, CD and SD review

–How to model the object's decision what to do when receiving a message using State Machine Diagrams and Activity Diagrams

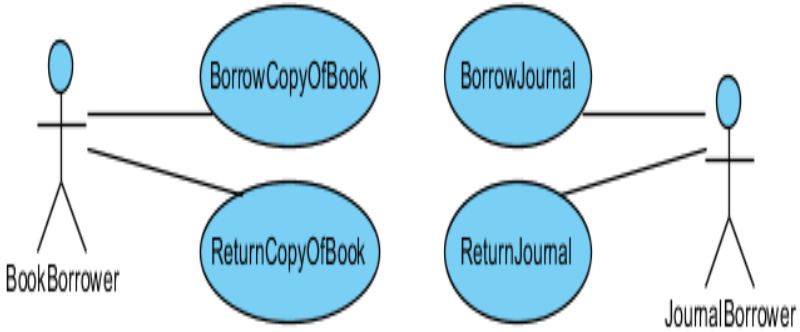
Learn what are the Symptoms of Rotting Design

Learn what are the SOLID principles of OOD and how they are used to Avoid the Symptoms


UML 2.4 Diagram
Structure Diagram
Behavior Diagram
Class Diagram
Object Diagram
Package Diagram
Model Diagram
Composite Structure Diagram
Component Diagram
Manifestation Diagram
Deployment Diagram
Network Architecture Diagram
Profile Diagram
UseCase Diagram
Activity Diagram
State Machine Diagram
Interaction Diagram
Sequence Diagram
Communication Diagram
Timing Diagram
Interaction Overview Diagram

Use Case Diagram (UCD)

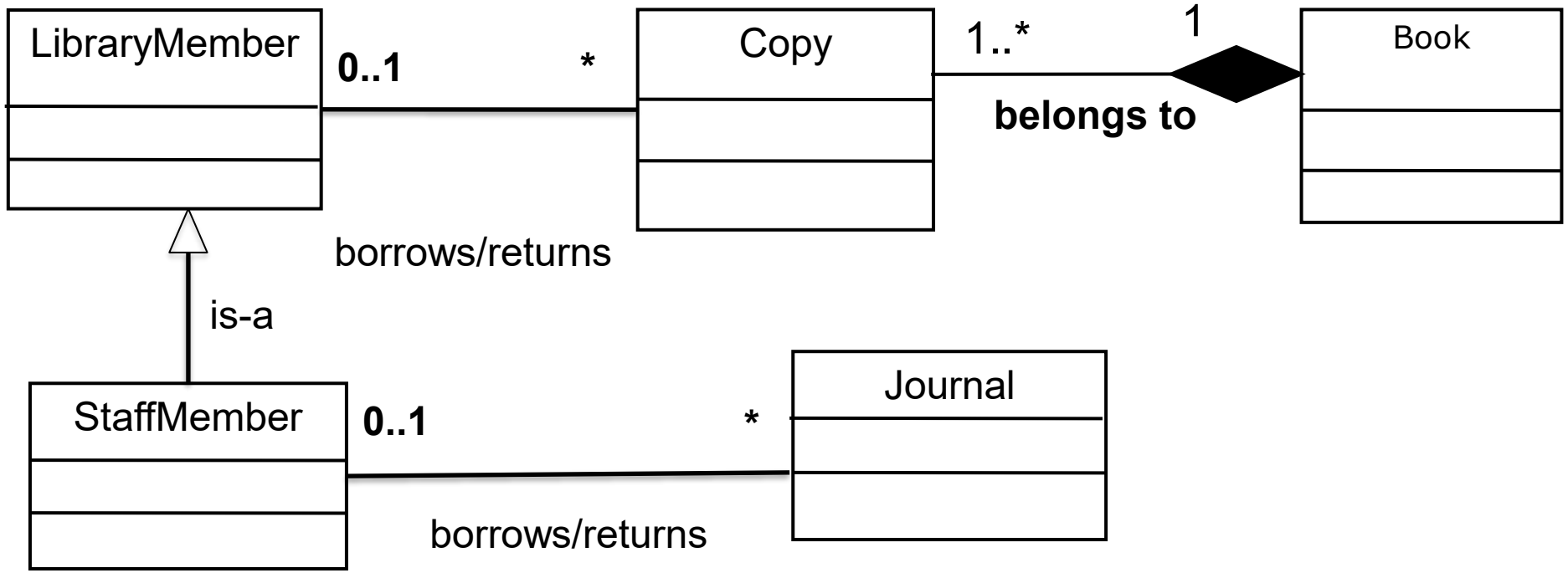
- A UML use case diagram is the primary form of system/software requirements for a new software program underdeveloped
- Use cases specify the expected behaviour (**what**), and **NOT** the exact method of making it happen (**how**)



Class Diagram (CD)

- Used for describing structure in the use cases
- A transition from "what" the system must do, to "how" the system will do it
  - What classes will we need to implement a system that meets our requirements?
  - What attributes and methods will each class have?
  - How will the classes interact with each other?

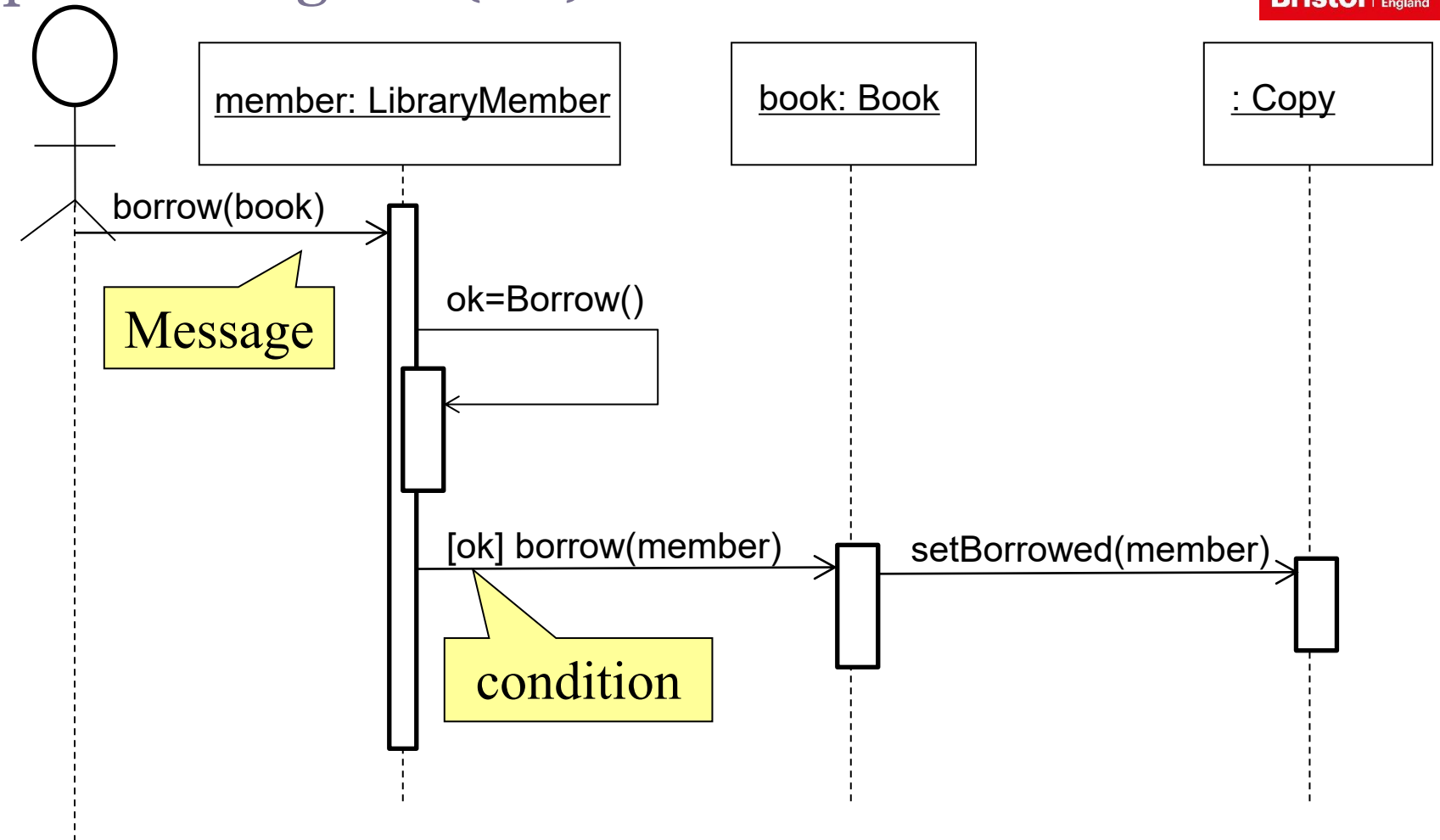
Class Diagram (CD)



Interaction Diagrams

- They model the behavior of use cases by describing the way groups of objects and entities interact to complete the task of the use case.
- They help us to visualize the interactive behaviour of a system.
- They represent how one or more objects in the system connect and communicate with each other.
- They are used to observe the dynamic behaviour of a system.

Sequence Diagram (SD)



State Machine Diagrams (SMDs)

UWE Bristol University of the West of England

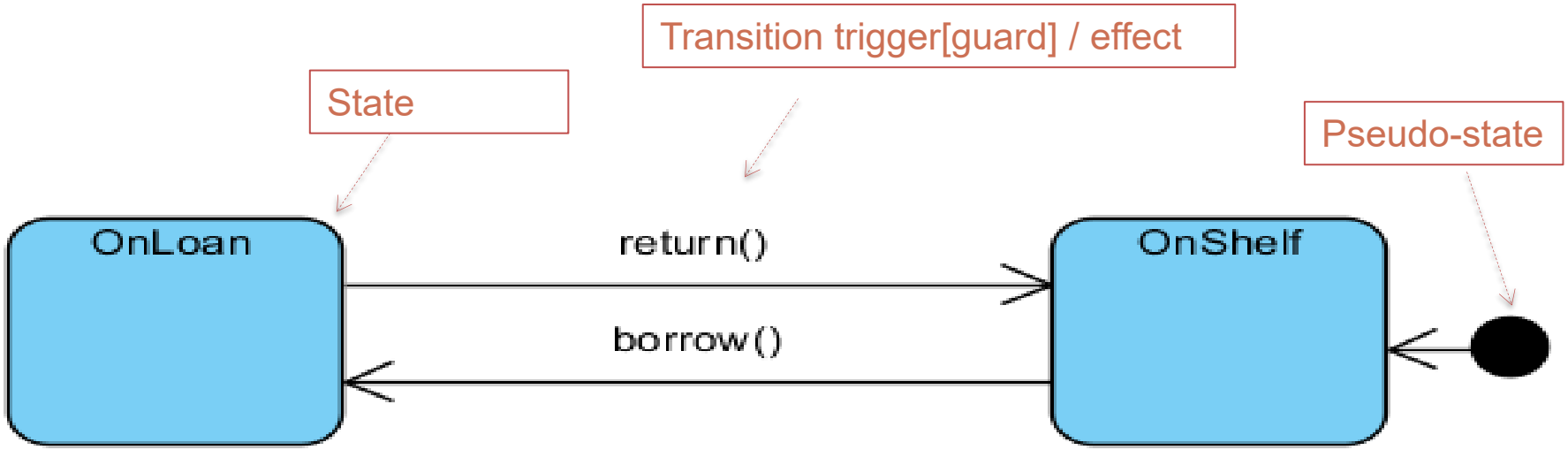
- In order to implement a class we need to understand what the dependencies are between the state of an object and its reaction to messages or other events

- **State diagrams**
  - show the states of a single object,
  - the events or the messages that cause a transition from one state to another and
  - the action that result from a state change.

- You do not have to create a SMD for every class!

State Machine Diagrams (SMDs) cont.

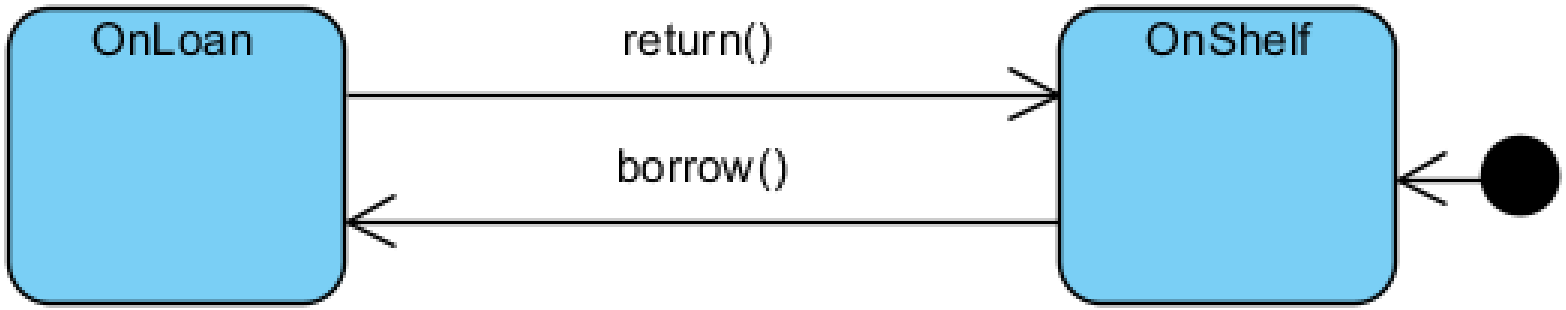
- *"A state is an abstraction of the attribute values and links of an object. Sets of values are grouped together into a state according to properties that affect the gross behavior of the object."* –Rumbaugh



Final/stop state

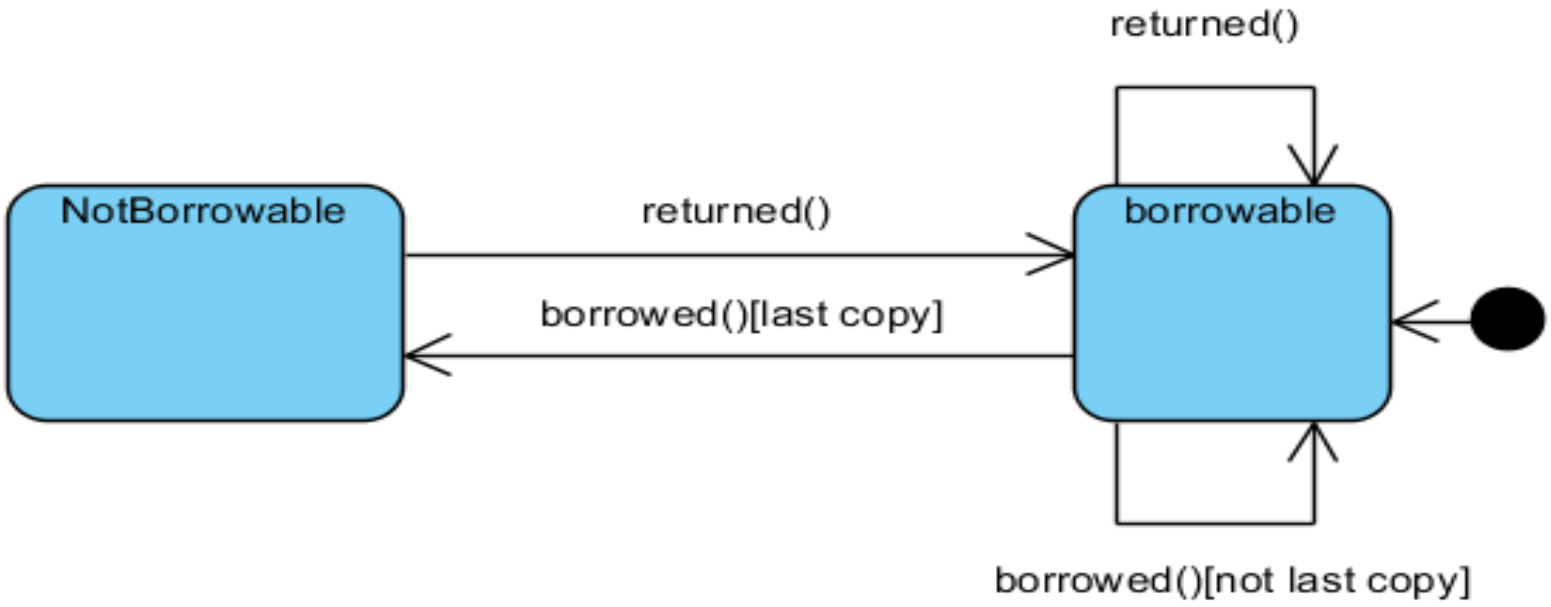
State Machine Diagrams (SMDs) cont.

- There are two special states
  - **Start state:** Each state diagram must have one and only one start state
  - **Stop State:** An object can have multiple stop states



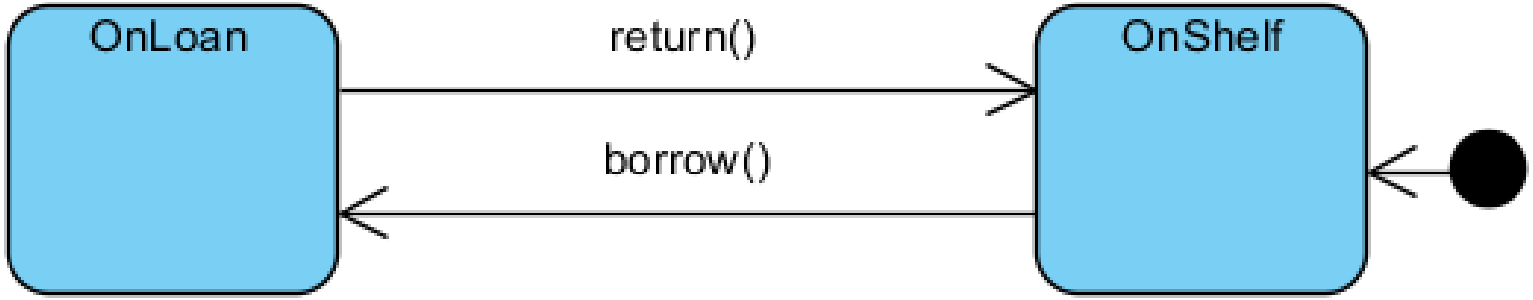
State Machine Diagrams (SMDs) cont.

- Guard
  - Sometimes a change of state of the object depends on the exact values of an object's attributes



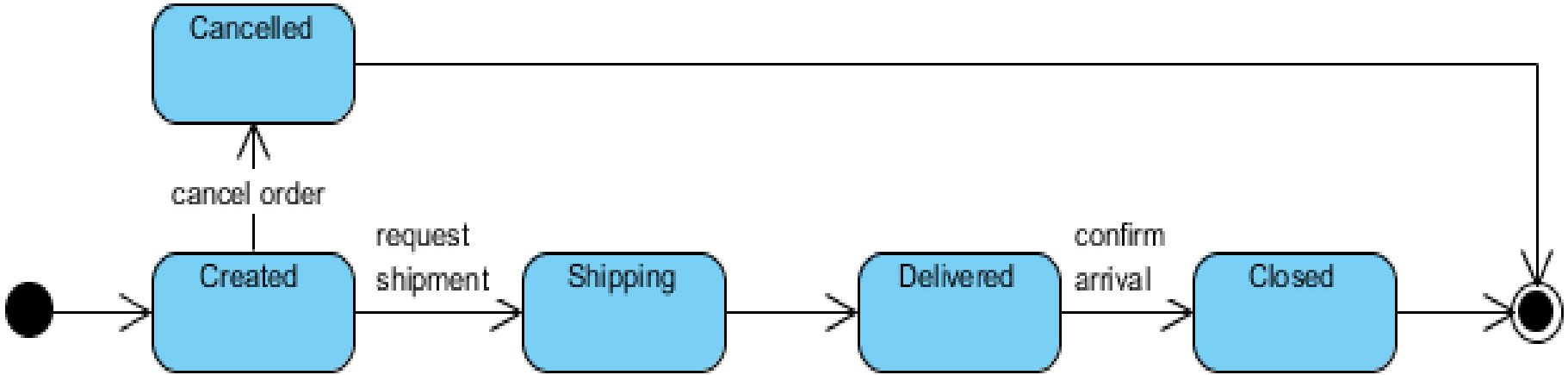
State Machine Diagrams (SMDs) cont.

- **Closed loop** state machine diagram does not have a final state
- The object lives until the entire system terminates



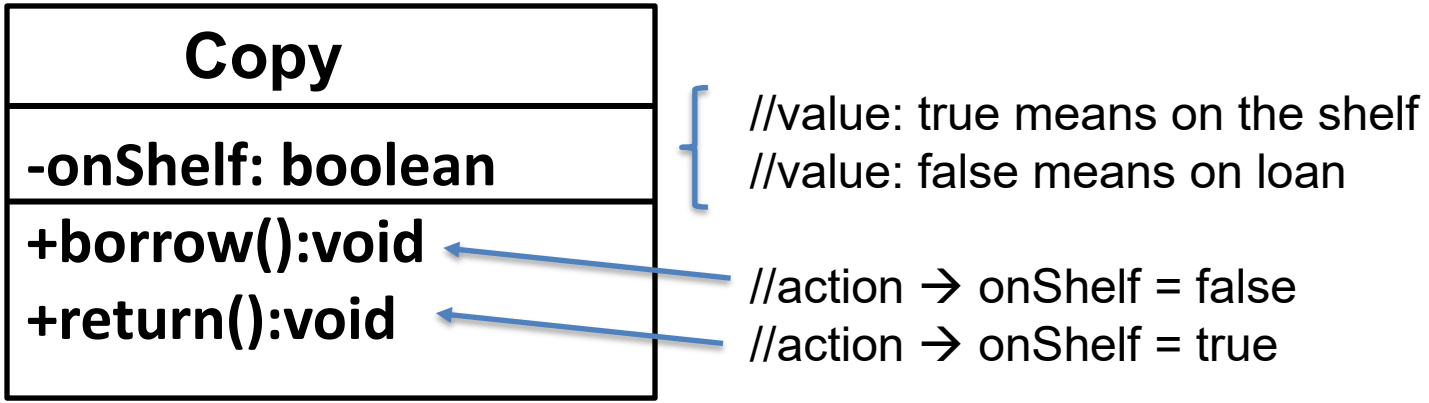
State Machine Diagrams (SMDs) cont.

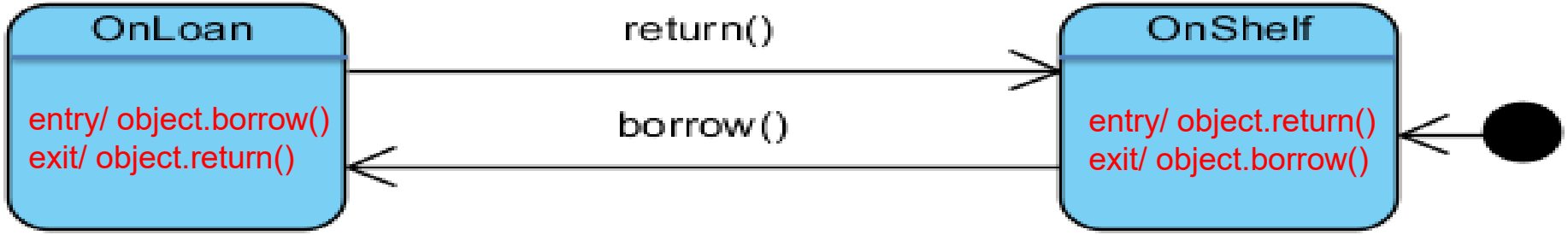
- **Open loop** state machine represents an object that may terminate before the system terminates



Source: Visual paragdigm

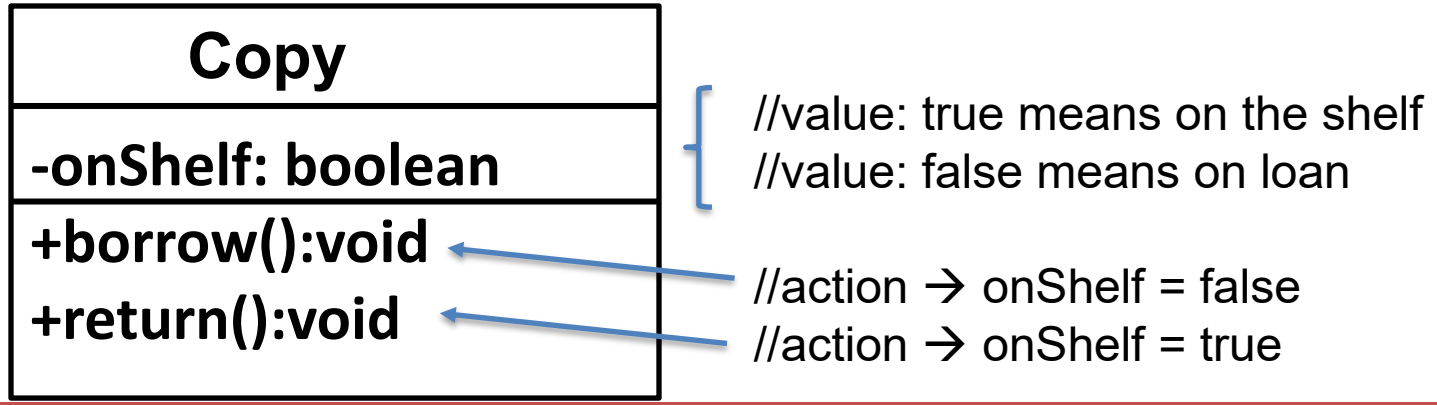
State Machine Diagrams (SMDs) cont.



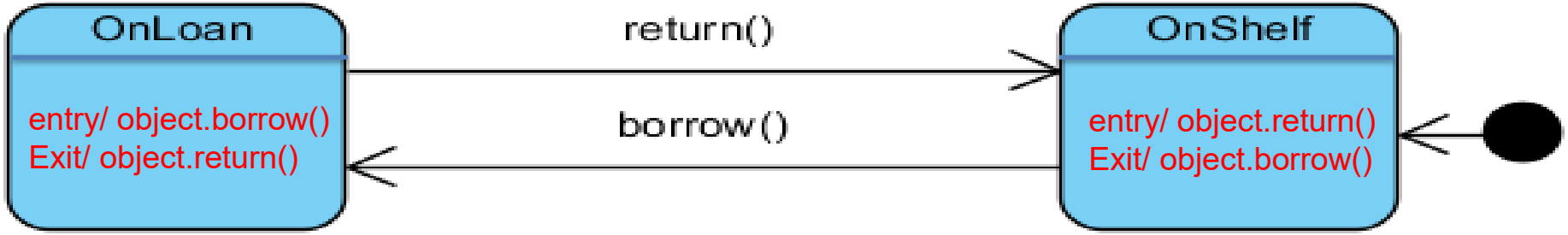


Source and Further reading entry/exit actions etc.: [Visual paragdigm]

State Machine Diagrams (SMDs) cont.



Can you build test cases from SMDs?



Source and Further reading entry/exit actions etc.: [Visual paragdigm]

Activity Diagrams (ADs)

Activity Diagrams (ADs) cont.

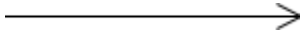
- ADs describe how activities are co-ordinated
- Support parallel behaviour (unlike flowcharts)

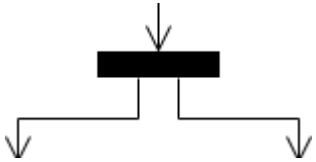
- Activity diagrams are recommended in the following situations:
  - Analysing use case
  - Dealing with multithreaded application
  - Understanding workflow across many use cases

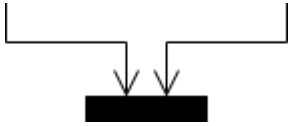
ADs have similarities with BPMN or RAD

Activity Diagrams (ADs) cont.

- ADs Components
  - Activities: Named box with rounded corners (you can think of an activity as a state that is left once the activity is finished)
  - Activity edge: Arrow (similar to transition but fires only when the previous activity completes)
  - Synchronisation bar: Thick horizontal line describing the co-ordination of activities
  - Decision diamond: Used to show decisions
  - Start and stop markers: Same as in state diagrams

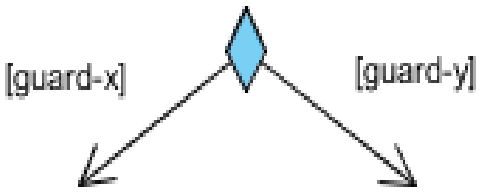




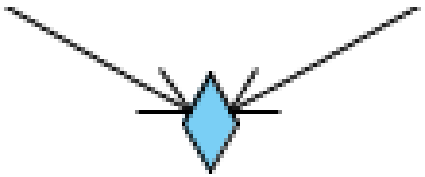


Activity Diagrams (ADs) cont.

- Decision Node and Merge Node
  - To show conditional behaviour use a branch and a merge
  - The top diamond is a branch and has only one transition flowing into it and any number of mutually exclusive transitions flowing out
  - That is, the guards on the outgoing transitions must resolve themselves so that only one is followed
  - The bottom diamond is a merge, it is used to end the conditional behaviour
  - There can be any number of incoming, and only one outgoing transition



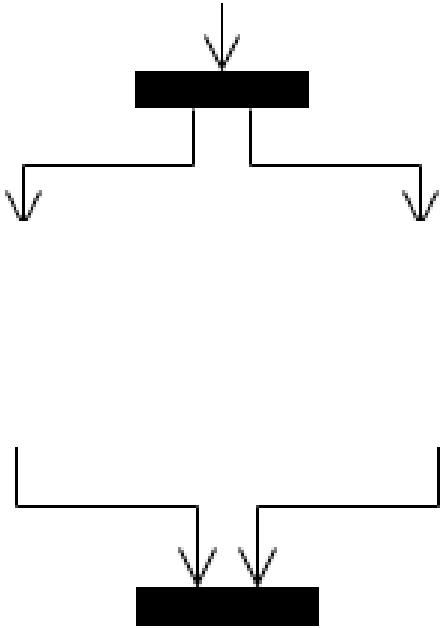
Decision Node



Merge Node

Activity Diagrams (ADs) cont.

- To show **parallel behaviour** use a fork and a join
  - The **fork** (top) has one transition entering and any number of transitions exiting, all of which will be taken
  - The **join** (bottom) represents the end of the parallel behaviour and has any number of transitions entering, and only one leaving



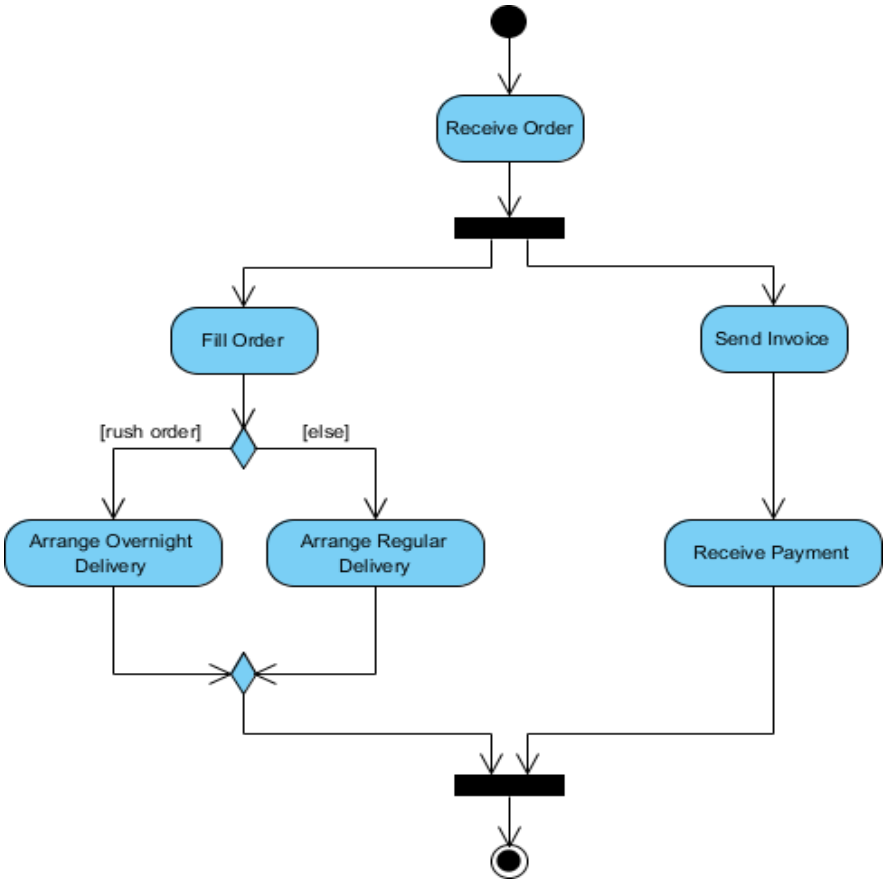
Activity Diagrams (ADs) cont.

- **Process Order - Problem Description**

"Once the order is received, the activities split into two parallel sets of activities. One side fills and sends the order while the other handles the billing.

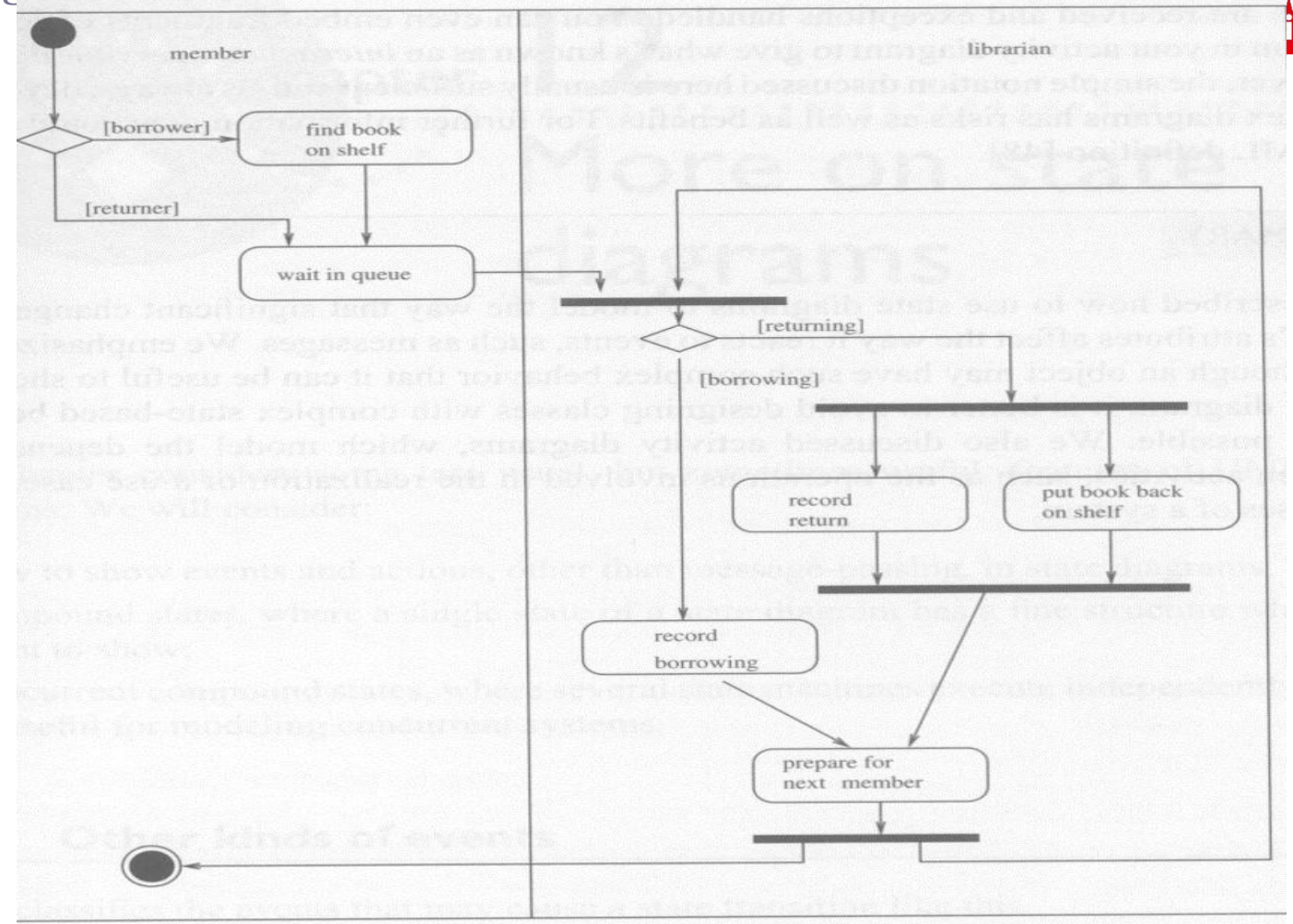
On the Fill Order side, the method of delivery is decided conditionally. Depending on the condition either the Overnight Delivery activity or the Regular Delivery activity is performed.

Finally the parallel activities combine to close the order."



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/

Activity Diagrams (ADs) cont. (Library Example)



Summary, Reference and suggested reading

- UML diagrams are very useful for staying organised during software development and for communication with team members or clients
- There are many different types of UML diagrams out there but usually people only use a very small subset (most often use case, class and sequence diagrams)

Suggested reading

UML Distilled, by Martin Fowler & Kendal Scott, Chapter 4
http://www.uml-diagrams.org/uml-24-diagrams.html
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-state-machine-diagram/
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/

Self study: **Component Diagrams:** https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/
Deployment Diagrams: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-deployment-diagram/

General OO Concepts

Three most general OO Concepts

UWE Bristol University of the West of England

- **Encapsulation:**
  - Internal representation of an object is hidden from outside world. This is also referred as information hiding. For example, a class encapsulates member variables and methods into a single unit 'class' and can restrict access to them e.g., private or protected members.
    - Encapsulation examples in Python: https://www.geeksforgeeks.org/encapsulation-in-python/
- **Inheritance:**
  - The relationship between classes whereby one class **inherits** all or part of the description of another, more general, class, and class instances inherit all the properties and methods of the classes they belong to. For example, Student is a Person; Staff is a Person and both inherit properties and methods from Person class.
    - Inheritance in Python example: https://www.w3schools.com/python/python_inheritance.asp

Three most general OO Concepts cont.

UWE Bristol University of the West of England

- **Polymorphism:**
  - One name multiple form, and the ability of an object or operator to refer to instances of different classes at run time
    - Shape has a method that calculates area;
    - Circle is a Shape and can use area calculation; Triangle is a Shape and can also use area calculation; Polymorphism allows to calculate area of Circle or Triangle through one object.
      - Polymorphism in Python example: https://www.geeksforgeeks.org/polymorphism-in-python/

        Polymorphic function – *len('Hello World')* or *len([20, 10, 'hello world'])*

Three most general OO Concepts cont.

ASD > Lab1 > polymorphism_v0.py > ...

```python
class Python():
    def language(self):
        print("This is Python")

    def type(self):
        print("supports sequential, functional and OOP")

class Java():
    def language(self):
        print("This is Java")

    def type(self):
        print("supports OOP")

def polymorphic_func(obj):
    obj.language()
    obj.type()

python_lang = Python()
java_lang = Java()
polymorphic_func(python_lang)
polymorphic_func(java_lang)
```

This is Python
supports sequential, functional and OOP
This is Java
supports OOP

Three most general OO Concepts cont.

ASD > Lab1 > polymorphism_v00.py > ...

```python
class Bird:
    def __init__(self, type):
        self.type = type

    def intro(self):
        print("There are many types of birds.")

    def flight(self):
        print("Most of the birds can fly but some cannot.")

class sparrow(Bird):
    def __init__(self, type, size):
        super().__init__(type)
        self.size = size

    def flight(self):   #overriding
        print(self.type, " can fly.")

    def getSize(self):
        print('Sparrow size is: ', str(self.size))

class ostrich(Bird):
    def __init__(self, type, speed):
        super().__init__(type)
        self.speed = speed

    def intro(self):    #overriding
        print("Ostrich are very unique birds.")

    def flight(self):   #overriding
        print(self.type, " cannot fly.")

    def getSpeed(self):
        print('Ostrich speed is: ', str(self.speed))
```

```python
obj_bird = Bird('Bird')
obj_spr = sparrow('Sparrow', 4)
obj_ost = ostrich('Ostrich', 20)

#One way to access methods
obj_bird.intro()
obj_bird.flight()

obj_spr.intro()
obj_spr.flight()
obj_spr.getSize()

obj_ost.intro()
obj_ost.flight()
obj_ost.getSpeed()

#Another way to access methods
print('\n# Checking bird objects in a loop \n')
for obj in [obj_bird, obj_spr, obj_ost]:
    obj.intro()
    obj.flight()
```

Output

```
There are many types of birds.
Most of the birds can fly but some cannot.
There are many types of birds.
Sparrow  can fly.
Sparrow size is:  4
Ostrich are very unique birds.
Ostrich  cannot fly.
Ostrich speed is:  20

# Checking bird objects in a loop

There are many types of birds.
Most of the birds can fly but some cannot.
There are many types of birds.
Sparrow  can fly.
Ostrich are very unique birds.
Ostrich  cannot fly.
```

Example adapted from:
https://www.geeksforgeeks.org/polymorphism-in-python/

Solid Principles

Symptoms of Rotting Design

UWE Bristol University of the West of England

- **Rigidity**
  - The design is hard to change. Every change causes a cascade of subsequent changes in dependent modules
  - **Telltale sign**: 'Huh, it was a lot more complicated than I thought.'
- **Fragility**
  - The design is easy to break. It is the tendency of the software to break in many places every time it is changed
  - the code breaks in unexpected places that have no conceptual relationship with the changed area, and fixing the problems causes new problems
  - **Telltale signs**: some modules are constantly on the bug list, time is used finding bugs, not fixing them
- **Immobility**
  - The design is hard to reuse. It is the inability to reuse software from other projects or from parts of the same project
  - **Telltale sign**: a module could be reused but the effort and risk of separating it from the original environment is too high

Symptoms of Rotting Design cont.

UWE Bristol University of the West of England

- **Viscosity:** refers to **the ease at which a developer can add design-preserving code to a system; and Viscosity of the design**: changes or additions are easier to implement by doing the wrong thing
  - **Telltale signs**: when a change is needed, programmers are tempted to hack rather than to preserve the original design
- **Fragmentation:** when functionality is scattered across many classes, or many libraries
- **Needless complexity**
  - Contains elements that aren't currently useful
- **Needless repetition**
  - System has lots of repeated code elements

Five Principles to Avoid the Symptoms

1. **S**ingle-Responsibility Principle (SRP)
2. **O**pen–Closed Principle (OCP)
3. **L**iskov Substitution Principle (LSP)
4. **I**nterface-Segregation Principle (ISP)
5. **D**ependency-Inversion Principle (DIP)

Known as **SOLID** principles of object oriented software design

1. Single-Responsibility Principle

UWE Bristol University of the West of England

A class should have single reason to change

- **Cohesion**: states how strongly-related and focused are the various responsibilities of a module or class
  - E.g., a component should implement a single logical function

- **Coupling**: the degree to which each program module relies on each one of the other modules
  - Shared data among modules, passing module control as parameters etc.

Less coupling and more cohesion is preferred (why?)

1. Single-Responsibility Principle cont.

- Class **Cohesive**
  - A class should model a single entity
  - A class should have a well defined role
  - Must avoid modelling multiple objects in a single class

- Why?
  - Readability, reusability, easy to use , easy to maintain

1. Single-Responsibility Principle cont.

- Example:

```
class UWECourse:
 def __init__(self, moduleCode, department, staffId, staffName):
  self.__ moduleCode = moduleCode
  self.__ department = department
  self.__ staffId = staffId
  self.__ staffName = staffName


 def getModuleCode(self):
   #body
 def getDepartment(self):
   #body
 def getStaffName(self):
   #body
```

Is there anything wrong with this class?

Low Cohesion class

1. Single-Responsibility Principle cont.

- Example of a high cohesive classes

```
class Staff:
 def __init__(self, staffId, staffName):
  self.__ staffId = staffId
  self.__ staffName = staffName

 def getStaffName(self):
   #body
```

```
class Department:
 def __init__(self, department):
  self.__ department = department

 def getDepartment(self):
   #body
```

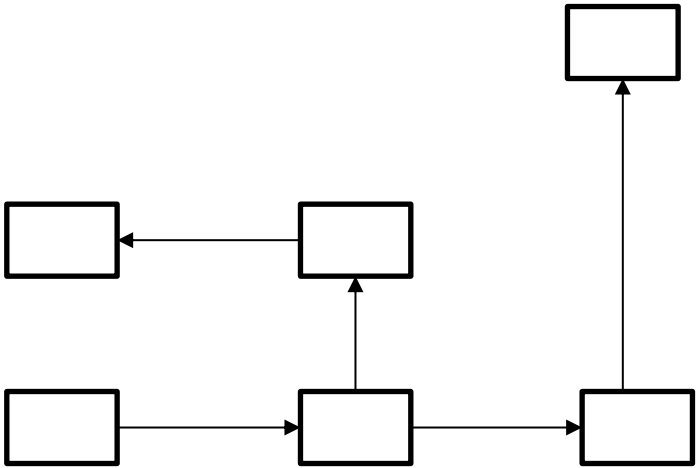
```
class Module:
 def __init__(self, moduleCode):
  self.__ moduleCode = moduleCode

 def getModuleCode(self):
   #body
```

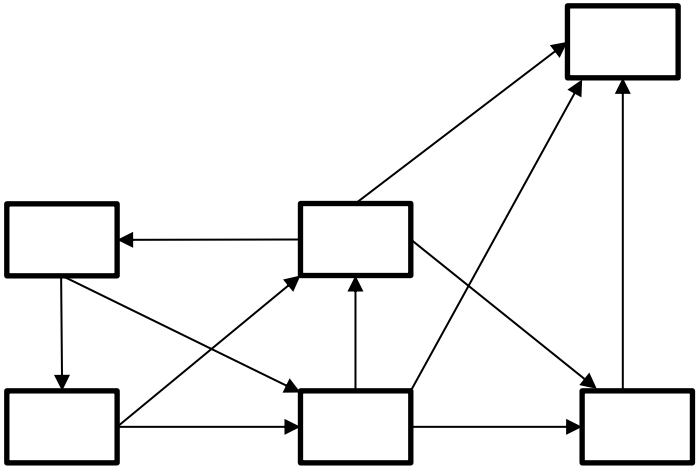
1. Single-Responsibility Principle cont.

- **Coupling**
  - Dependency of one class on another class to complete its job
  - Aim is for loose coupling
  - Tight coupling can make the program brittle and complex

1. Single-Responsibility Principle cont.

- **Coupling**





Which one of these look tight coupling?

Loose coupling

Tight coupling

1. Single-Responsibility Principle cont.

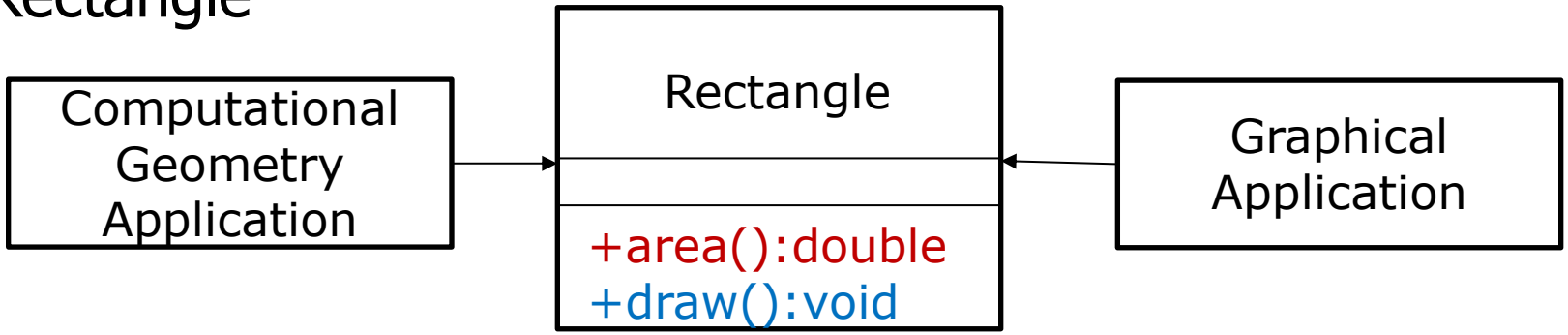
UWE Bristol University of the West of England

- A class should have single reason to change
- Responsibility = a reason to change

- Responsibilities are Axes of change
- A cohesive responsibility is a single axis of change
- A class should have only one responsibility

1. Single-Responsibility Principle cont.

- Rectangle



Dependency

1. Is SRP followed in this design?

2. Do you see any problem in the above design?

1. Single-Responsibility Principle cont.

UWE Bristol University of the West of England

- The Rectangle class has **two methods** shown.
  - One draws the rectangle on the screen, the other computes the area of the rectangle
- Two different applications use the Rectangle class.
  - One application does computational geometry. It uses Rectangle to help it with the mathematics of geometric shapes. It never draws the rectangle on the screen.
  - The other application is graphical in nature. It may also do some computational geometry, but it definitely draws the rectangle on the screen.
- This design violates the SRP. The Rectangle class has two responsibilities
  - The first responsibility is to provide a mathematical model of the geometry of a rectangle
  - The second responsibility is to render the rectangle on a graphical user interface

1. Single-Responsibility Principle cont.

UWE Bristol University of the West of England

- The violation of SRP causes several nasty problems

- **Firstly,** if a change to the GraphicalApplication causes the Rectangle to change for some reason, that change may force us to rebuild, retest, and redeploy the ComputationalGeometryApplication. If we forget to do this, that application may break in unpredictable ways

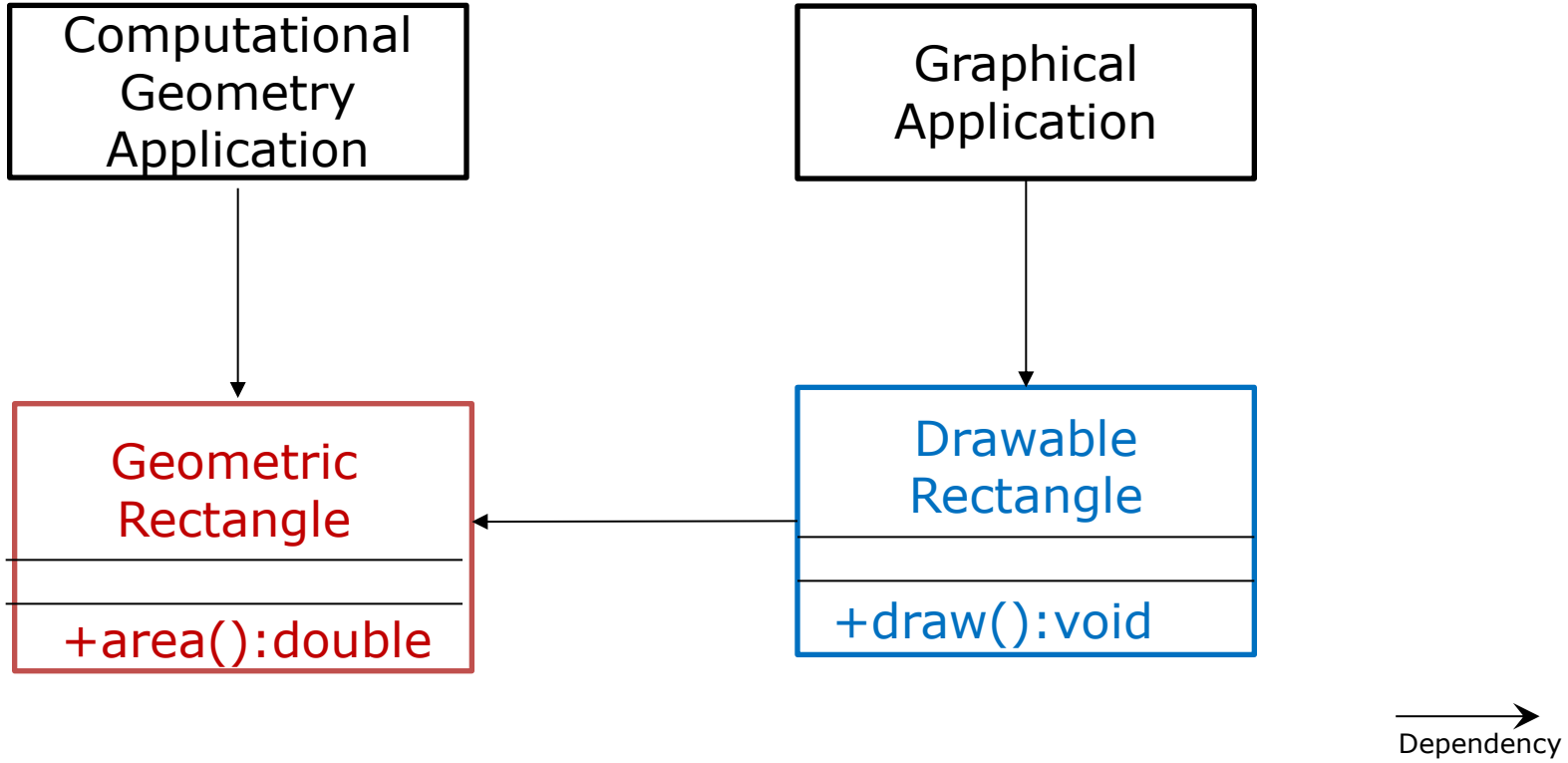
- **Secondly,** we must include the GUI in the computational geometry application
  - If this were a C++ application, the GUI would have to be linked in, consuming link time, compile time, and memory footprint
  - In a Java application, the *.class* files for the GUI have to be deployed to the target platform

1. Single-Responsibility Principle cont.

- Can you suggest a better Rectangle class design?

1. Single-Responsibility Principle cont.

- A better design



2. Open–Closed Principle

UWE Bristol University of the West of England

"Software entities should be open for extension, but closed for modification"– Bertrand Meyer

- 'Open for extension': the behavior of a module can be extended with new behaviors to satisfy the changing requirements (e.g., by sub-classing)
- 'Closed for modification': extending the module must not result in changes to the source or even binary code of the module

- A principle which states that we should add new functionality by adding new code, not by editing old code.

Open version (bad design)

```
class DiscountCalc:
 def __init__(self,productType, cost):
    self.productType = productType
    self.cost = cost

def getDiscountedPrice(self):
    if self.productType == 1:
       return self.cost - (self.cost*0.10)
    elif self.productType == 2:
       return self.cost - (self.cost*0.20)
    elif self.productType == 3:
       return self.cost - (self.cost*0.30)

dc_product1 = DiscountCalc (1, 100)
print(dc_product1.getDiscountedPrice())
dc_product2 = DiscountCalc (2, 100)
print(dc_product2.getDiscountedPrice())
dc_product3 = DiscountCalc (3, 100)
print(dc_product3.getDiscountedPrice())
```

Can you guess why is this bad design?

Can you suggest a better design?

2. Open–Closed Principle cont.

UWE Bristol University of the West of England

What is wrong with the code?

- `getDiscountedPrice`() is not closed.
  - *If/else* tend to recur in diverse places.
  - If we add a new product, we add to the if/else
  - All *if/else* statements must be found and edited.
  - *for each new product added the unit testing of the* `DiscountCalc` *should be redone*
- The software is both rigid and brittle

Open/closed version (good design)

```
from abc import ABC, abstractmethod
class DiscountCalc (ABC):
  @abstractmethod
  def getDiscountedPrice(self):
      pass

class DiscountCalcProduct1(DiscountCalc):
   def __init__(self,cost):
      self.cost = cost

   def getDiscountedPrice(self):
          return self.cost - (self.cost*0.10)
```

Open/closed version (good design)

```
class DiscountCalcProduct2(DiscountCalc):
   def __init__(self,cost):
      self.cost = cost

   def getDiscountedPrice(self):
         return self.cost - (self.cost*0.20)



class DiscountCalcProduct3(DiscountCalc):
   def __init__(self,cost):
      self.cost = cost

   def getDiscountedPrice(self):
         return self.cost - (self.cost*0.30)
```

Open/closed version (good design)

```
dc_product1 = DiscountCalcProduct1 (100)
print(dc_product1. getDiscountedPrice())

dc_product2 = DiscountCalcProduct2 (100)
print(dc_product2. getDiscountedPrice())

dc_product3 = DiscountCalcProduct3 (100)
print(dc_product3. getDiscountedPrice())
```

3. Liskov Substitution Principle (LSP)

UWE Bristol University of the West of England

- The LSP states that Subtypes must be substitutable for their base types. [Named for Barbara Liskov, who first described the principle in 1988]
- Derived classes must be usable through the base class interface, without the need for the user to know the difference.
- All derived classes must be substitutable for their base classes
- This principle guides us in the creation of abstractions

An example (Violating LSP)

```
class Vehicle:
 def __init__(self, name, speed):
     self.name = name
     self.speed = speed
 def get_name(self):
      return self.name
 def get_speed(self):
      return self.speed
 def engine(self):
       pass
 def start_engine(self):
      self.engine()
```

```
class Car(Vehicle):
   def start_engine(self):
       pass


class Bicycle(Vehicle):
   def start_engine(self):
       pass
```

Can you guess what is wrong with this code?

An example (Violating LSP)

```
class Car(Vehicle):
    def start_engine(self):
        pass


class Bicycle(Vehicle):
    def start_engine(self):
        pass
```

Bicycle class violates the LSP. The Vehicle class has an engine method, but a bicycle has no engine. So we cannot start any engine considering a Bicycle object.

if B(icycle) is a subtype of V(ehicle), then objects of type V may be replaced by objects of type B, without breaking the program.

An example (Conforming LSP): Code refactored

```
class Vehicle:
 def __init__(self, name, speed):
     self.name = name
     self.speed = speed
 def get_name(self):
      return self.name
 def get_speed(self):
      return self.speed
```

An example (Conforming LSP): Code refactored

```
class VehicleWithEngine(Vehicle):
    def engine(self):
        pass
    def start_engine(self):
        self.engine()


class VehicleWithoutEngine(Vehicle):
    def start_moving(self):
         raise NotImplemented
```

An example (Conforming LSP): Code refactored

```
class Car(VehicleWithEngine):
    def start_engine(self):
        pass



class Bicycle(VehicleWithoutEngine):
    def start_moving(self):
         pass
```

Self-study: Check another example at Pythontutorials.

4. Interface-Segregation Principle (ISP)

UWE Bristol University of the West of England

"Clients should not be forced to depend upon interfaces that they do not use."

"Many client-specific interfaces are better than one general purpose interface". — Robert Martin

- If you have a class with several different uses, create separate (narrow) interfaces for each use.
  - The alternative is that all uses share the same, large interface (and most uses will need only a small number of the methods.)
- Instead the uses of the class can be organized as groups of related methods, each of which serves a different purpose. Each of these groups becomes a separate interface.
- The purpose is to make clients use as small and as coherent an interface as possible.
- Fat interfaces lead to inadvertent couplings and accidental dependencies between classes.

bad example (polluted interface)

```
class Animal:
 @abstractmethod
 def walk(self):
   pass
@abstractmethod
 def eat(self):
   pass
```

bad example (polluted interface)

```
class Cat(Animal):
  def walk(self):
   #concrete implementation
  def eat(self):
  #concrete implementation
```

```
class Dog(Animal):
  def walk(self):
   #concrete implementation
  def eat(self):
  #concrete implementation
```

bad example (polluted interface)

```
class Snake(Animal):
  def walk(self):
   #concrete implementation NOT APPLICABLE
  def eat(self):
  #concrete implementation
```

bad example (polluted interface)

```
class Snake(Animal):
  def walk(self):
   #concrete implementation NOT APPLICABLE
  def eat(self):
  #concrete implementation


   def slither(self):
    #this behaviour needed
```

Solution conforming ISP

– split into two interfaces

```
class WalkableAnimal:
 @abstractmethod
 def walk(self):
   pass
@abstractmethod
 def eat(self):
   pass
```

```
class SlitherAnimal:
 @abstractmethod
 def slither(self):
   pass
@abstractmethod
 def eat(self):
   pass
```

5. Dependency-Inversion Principle (DIP)

UWE Bristol University of the West of England

Depend upon abstractions. Do not depend upon concrete implementations *(originally defined by Martin).*

- High level classes should not depend on low level classes.
- Abstractions should not depend upon the details. *Details should depend on abstractions.*
- If the high level abstractions depend on the low level implementation details, then the dependency is inverted from what it should be.
- High level classes should contain the "business logic", low level classes should contain the details of the (current) implementation.
- Access instances using interfaces or abstract classes to avoid dependency inversion.
- Open-Closed Principle and Liskov Substitution Principle => implicitly aligned to be compliant to the Dependency Inversion Principle

Violating DIP

```
class BackendDeveloper:
  #This is a low-level module
  def python():
   print("Writing Python code")

class FrontendDeveloper:
   #This is a low-level module
   def javascript():
     print("Writing JavaScript code")
```

Violating DIP

```
class Project:
 #This is a high-level module
  def __init__(self):
    self.backend = BackendDeveloper()
    self.frontend = FrontendDeveloper()


  def develop(self):
    self.backend.python()
    self.frontend.javascript()
    return "Develop codebase”
project = Project()
print(project.develop())
```

#The “Project” module calls the other two

What’s the problem in the above design?

5. Dependency-Inversion Principle (DIP)

UWE Bristol University of the West of England

- The two low level modules "`BackendDeveloper`" and "`FrontendDeveloper`" are reusable and they can be used in many other programs to gain access to the code and the interface

- However the "Project" module is not reusable in any context which does not involve a `BackendDeveloper` or a `FrontendDeveloper`

5. Dependency-Inversion Principle (DIP)

UWE Bristol University of the West of England

- Consider a new requirement: use of Database (as a low-level module)
- As we extend the code, Project module should be re used, but it needs a modification, adds new interdependencies to the system
- As time goes on, and more and more low-level modules participate in the program, the "Project" module will be littered with additional statements and will be dependent upon many lower level modules
- It will eventually become rigid and fragile

Conforming DIP-code refactored

```
class BackendDeveloper:
    #This is a low-level module
    def develop(self):
        self.__python_code()
    def __python_code():
        print("Writing Python code")
class FrontendDeveloper:
    #This is a low-level module
    def develop(self):
        self.__javascript()
    def __javascript():
        print("Writing JavaScript code")
```

Conforming DIP-code refactored

```
class Developers:
    #An Abstract module
    def __init__(self):
        self.backend = BackendDeveloper()
        self.frontend = FrontendDeveloper()
    def develop(self):
        self.backend.develop()
        self.frontend.develop()
```

Conforming DIP-code refactored

```
class Project:
    #This is a high-level module
    def __init__(self):
        self.__developers = Developers()
    def develops(self):
        return self.__developers.develop()

project = Project()
print(project.develops())
```

SOLID principles-Summary

UWE Bristol University of the West of England

- The Single Responsibility Principle is about actors and high level architecture
- The Open/Closed Principle is about class design and feature extensions
- The Liskov Substitution Principle is about subtyping and inheritance
- The Interface Segregation Principle (ISP) is about business logic to clients communication
- The dependency inversion principle is about decoupling software modules

References

- **SOLID Principles (with Python)**
  - https://www.oodesign.com/design-principles
  - https://medium.com/@vubon.roy/solid-principles-with-python-examples-10e1f3d91259
  - https://towardsdatascience.com/solid-coding-in-python-1281392a6a94
  - https://www.hashbangcode.com/article/solid-principles-python

- **Next Lecture**

–Design Patterns